

Homework 5

Bryce Davis

August 2026

1 Problem and Data

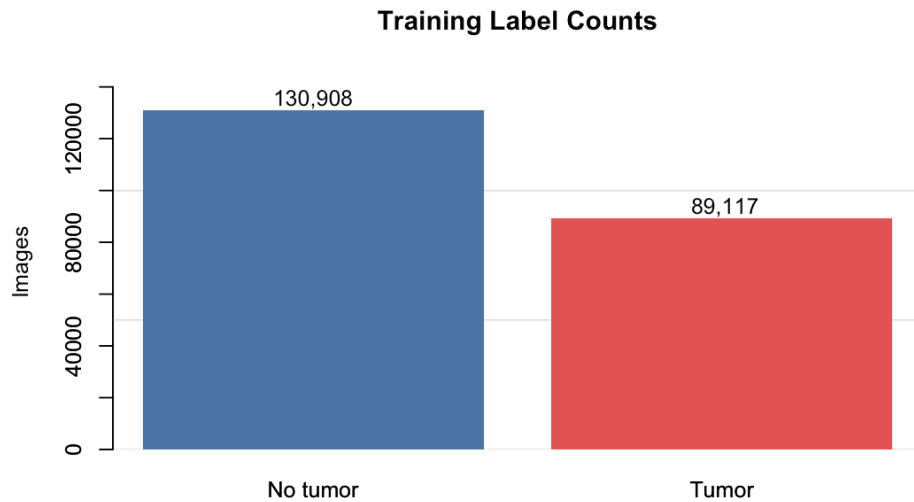
The goal of the Histopathologic Cancer Detection competition is to identify tumor tissue in small pathology images. Each image is a 96 by 96 pixel TIFF patch from a lymph node section. A positive label means that the center 32 by 32 pixel area contains at least one pixel of tumor tissue. The task is binary image classification, and Kaggle scores submissions with ROC AUC.

The dataset has 220,025 labeled training images and 57,458 unlabeled test images. The label file contains an image ID and a binary label for every training image. The full download is 7.76 GB. I used the competition data directly in a Kaggle notebook so that the image files did not need to be added to GitHub.

I checked the label file for duplicate IDs and checked that every listed training image had a matching file. There were no duplicate IDs and no missing training files. I also fixed the random seed at 42 so that the sample and train-validation split could be repeated. I did not remove any images because Kaggle states that duplicate patches from the original PCam data were already removed from this version.

2 Exploratory Data Analysis

The training set contains 130,908 images without tumor tissue and 89,117 images with tumor tissue. This is about 59.5 percent negative and 40.5 percent positive. The classes are not equal, but both groups are large enough for training. I used a stratified split so the model and validation sets kept about the same class ratio.



I displayed five example patches from each class in the notebook. The colors and tissue patterns varied within both classes, and the tumor patches were not separated by one obvious visual feature. I also compared mean pixel brightness for a random sample of 2,000 images. The two brightness distributions overlapped, so brightness alone would not be a useful classifier. This supported using a convolutional model that could learn spatial patterns in the images.

For training, I selected a reproducible stratified sample of 15,000 labeled images. I used 12,000 for training and 3,000 for validation. I resized the patches to 64 by 64 pixels and scaled pixel values to the range from 0 to 1. Using a subset and smaller image size kept the run short enough for the available Kaggle session while still leaving a useful validation set.

3 Model Architecture

I used a small convolutional neural network because the input is image data and local tissue patterns matter. The network has convolutional layers with 32, 64, and 128 filters. Each convolution is followed by max pooling. Global average pooling reduces the last feature maps, dropout limits overfitting, and a sigmoid output gives the probability of the tumor class. The model uses binary cross-entropy loss and the Adam optimizer.

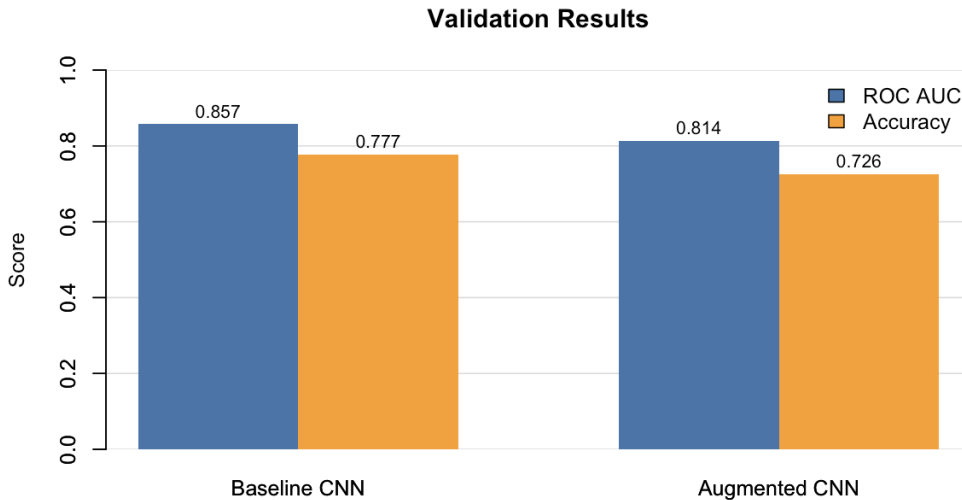
I first trained a baseline model for two epochs with a learning rate of 0.001 and dropout of 0.30. I then tested one adjusted model with random flips, small rotations, a learning rate of 0.0005, and dropout of 0.40. The second run was meant to check whether basic augmentation improved validation performance. Early stopping monitored validation ROC AUC and restored the best weights.

Setting	Baseline CNN	Augmented CNN
Training images	12,000	12,000
Validation images	3,000	3,000
Learning rate	0.001	0.0005
Dropout	0.30	0.40
Augmentation	None	Flips and rotations
Maximum epochs	2	3

4 Results and Analysis

Kaggle uses ROC AUC, so I treated validation ROC AUC as the main measure. I also reported accuracy at a probability cutoff of 0.5 because it is easy to interpret, but it was not used to select the submission model.

Model	Validation ROC AUC	Validation accuracy
Baseline CNN	0.857	0.777
Augmented CNN	0.814	0.726



The baseline CNN performed better on both measures. Its validation ROC AUC was 0.857, compared with 0.814 for the augmented model. The adjusted model also stopped after its validation AUC failed to improve. In this short run, the extra image changes and stronger dropout made learning harder instead of improving generalization. More epochs or milder augmentation might change that result, but the current validation results support using the baseline model.

I used the baseline CNN to predict probabilities for all 57,458 test images. The resulting submission file had the required columns named id and label and one row for each test image. The Kaggle submission received a score of 0.8241.

The screenshot shows a Kaggle competition notebook interface. At the top, the title 'HW5 Histopathologic Cancer Detection' is displayed. Below the title, there are tabs for 'Notebook', 'Input', 'Output', 'Logs', 'Comments (0)', and 'Settings'. The notebook is part of a 'Competition Notebook' for 'Histopathologic Cancer Detection'. It shows a 'Private Score' of 0.8241 and a 'Best Score' of 0.8241 V2. The notebook is titled 'Setup and exploratory data analysis'. The code cell shows imports for various libraries: os, random, pathlib, cv2, matplotlib.pyplot, numpy, pandas, seaborn, tensorflow, sklearn.metrics, and sklearn.model_selection. It also sets a seed, image size, and batch size. On the right side, there are sections for 'Runtime' (11m 19s - GPU P100), 'Input' (COMPETITIONS: Histopathologic Cancer Detection), 'Tags' (GPU), 'Language' (Python), and a 'Table of Contents' with links to 'Setup and exploratory data analysis', 'Train and validation data', 'CNN models', 'Validation results', and 'Kaggle submission'.

The main limitation is that I trained on 15,000 of the 220,025 labeled images. I also resized the images and used only one train-validation split. A larger run could use more of the training set, test milder augmentation, and compare several learning rates. Patient-level information was not included, so I could not check whether visually related patches from the same source appeared in both parts of the split.

5 Conclusion

The small baseline CNN produced a useful first result without a long training run. It reached a validation ROC AUC of 0.857 and outperformed the augmented version. The comparison also showed that augmentation does not automatically help when the model is trained for only a few epochs. If I continued, I would train the baseline on more images first, then make one tuning change at a time and keep the same validation split for comparison.

6 Links and References

GitHub repository: <https://github.com/Bryced24/cspb-3202-hw5-cancer-detection>

Kaggle notebook: <https://www.kaggle.com/code/bryced0924/hw5-histopathologic-cancer-detection>

Kaggle competition and data: <https://www.kaggle.com/competitions/histopathologic-cancer-detection>

TensorFlow Keras API: https://www.tensorflow.org/api_docs/python/tf/keras

Scikit-learn ROC AUC documentation: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.roc_auc_score.html